Data Structures

BS (CS) _Fall_2025

Lab_01 Manual

Learning Objectives:

1. Installation
2. Templates
3. Templates with unit testing (Gtest)

National University of computer and emerging sciences

Visual Studio Installation and Setup

Installation and setup guide for C++ visual studio:

Step 1: Download Visual Studio community 2022

Address: https://visualstudio.microsoft.com/vs/community/

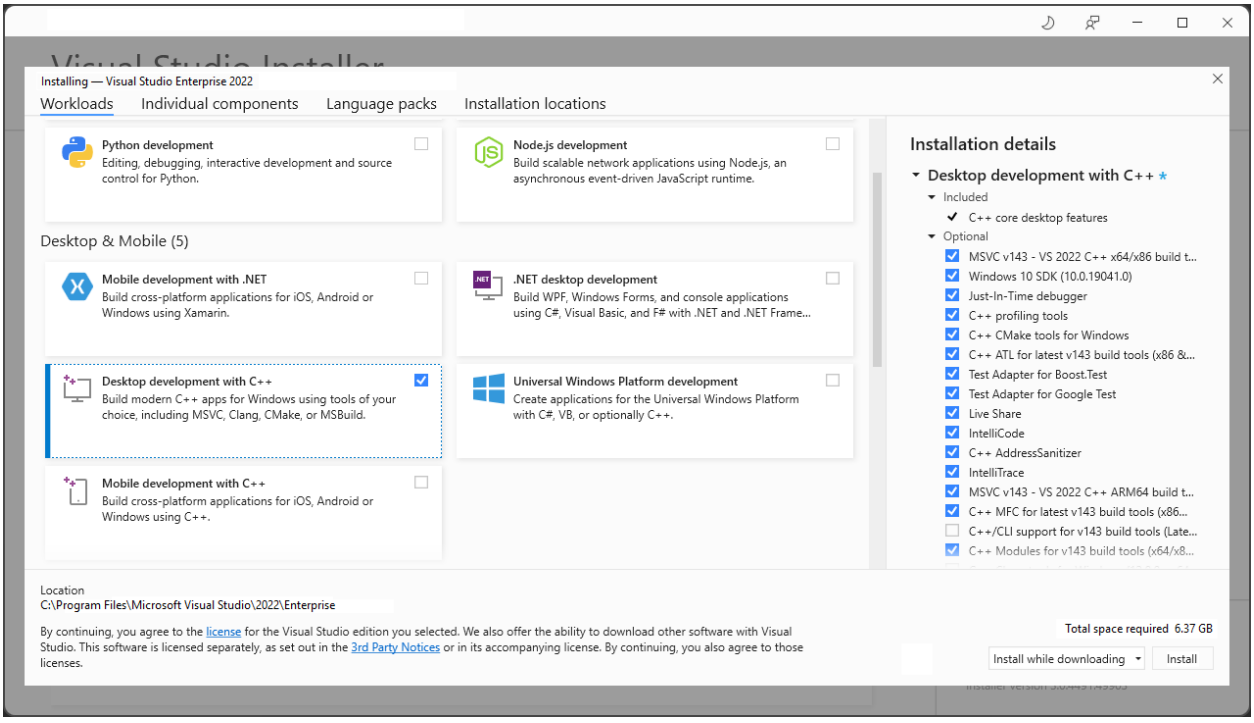
Step 2: Install the Visual Studio Installer

From your Downloads folder, double-click the bootstrapper file called VisualStudioSetup.exe.

If you receive a User Account Control notice, choose Yes to allow the bootstrapper to run.

Step 3: Choose workloads

You can use the installer to customize your installation by selecting the workloads, or feature sets, that you want. Select the Desktop Development with C++ and install.



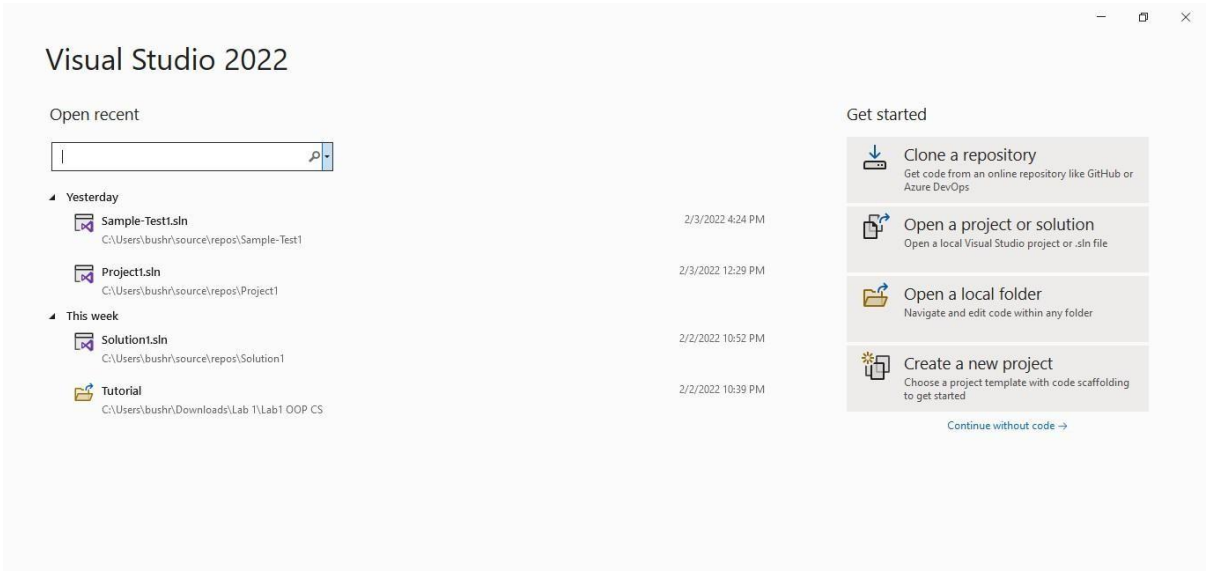
Screenshot of the Visual Studio 2022 installer with the Desktop development with C++ workload selected.

National University of computer and emerging sciences

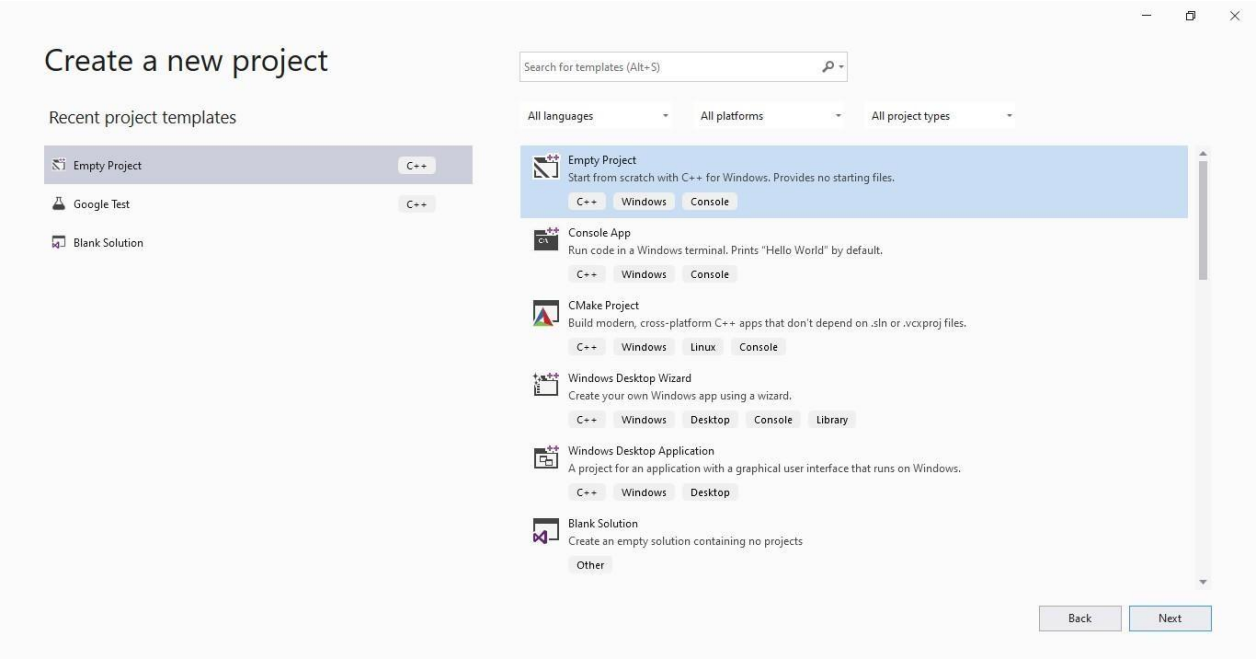
Getting started with Visual Studio

The usual starting point for a C++ programmer is a "Hello, world!" application that runs on the command line. That's what you'll create first in Visual Studio, and then we'll move on to something more challenging

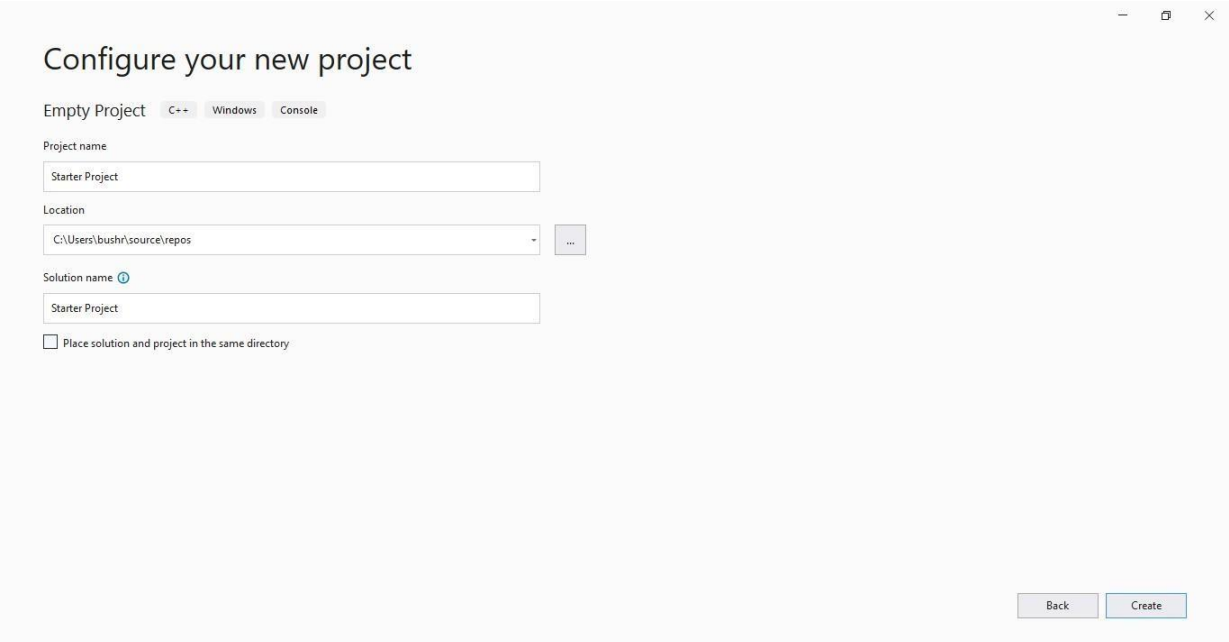
1. Choose Create a new project to get started.



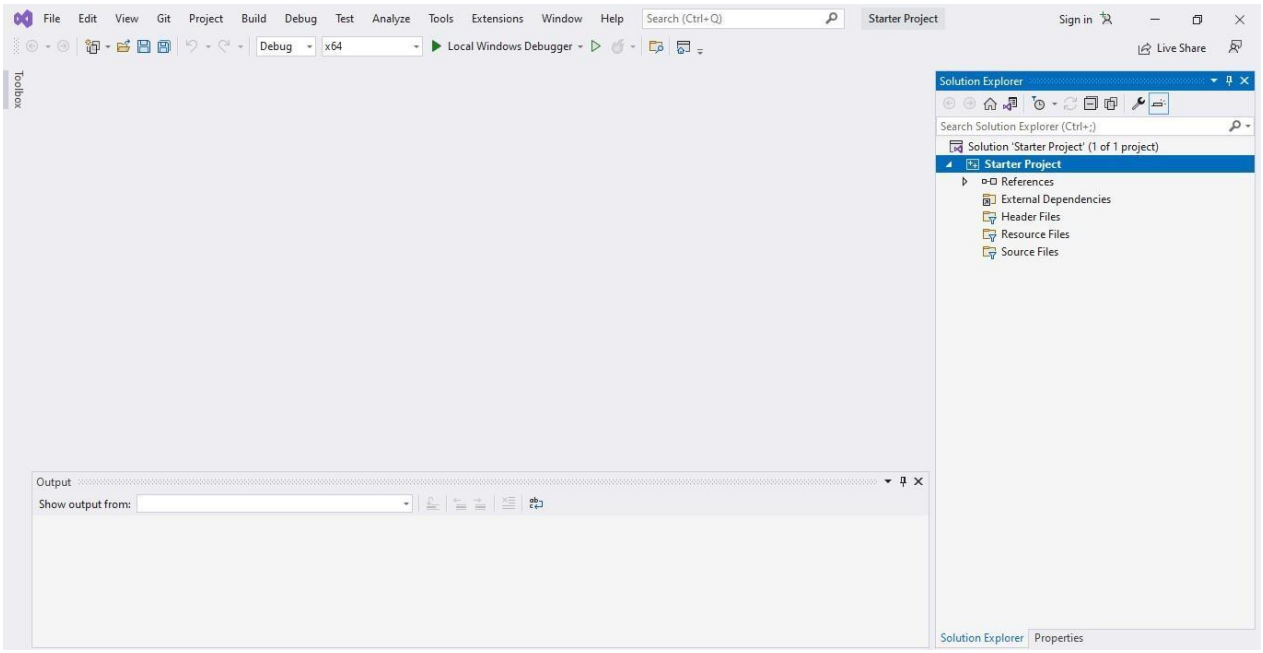
2. In the list of project templates, choose Empty Project, then choose Next.



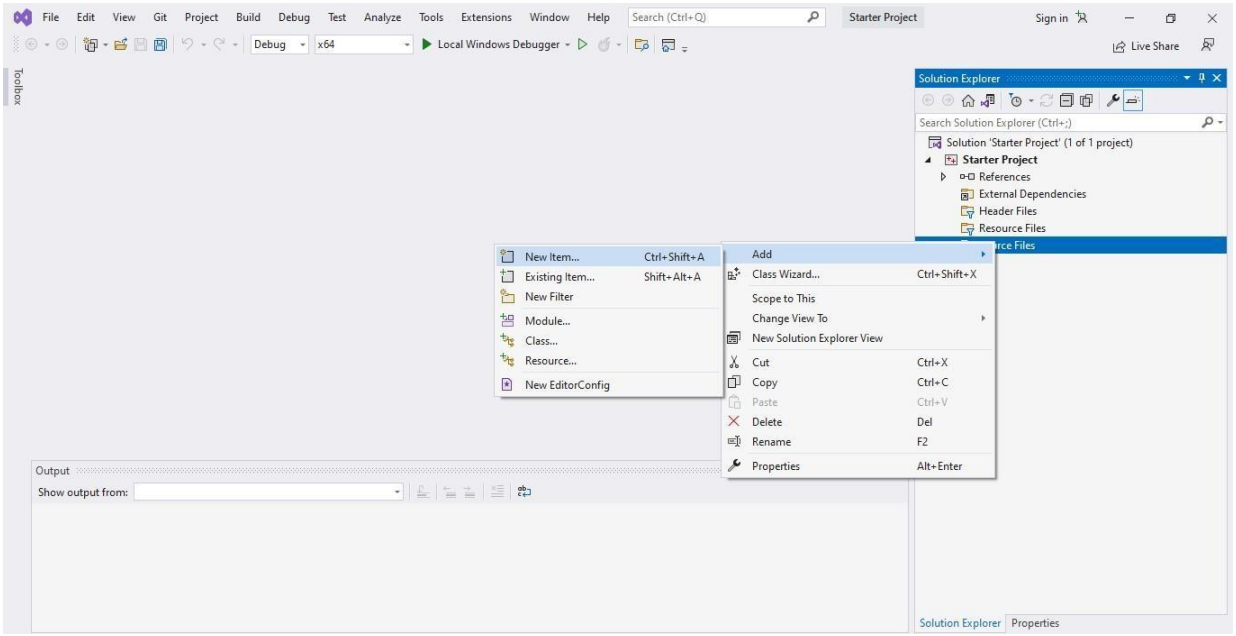
3. In the Configure your new project dialog box, select the Project Name edit box, name your new project Starter Project, then choose to Create.



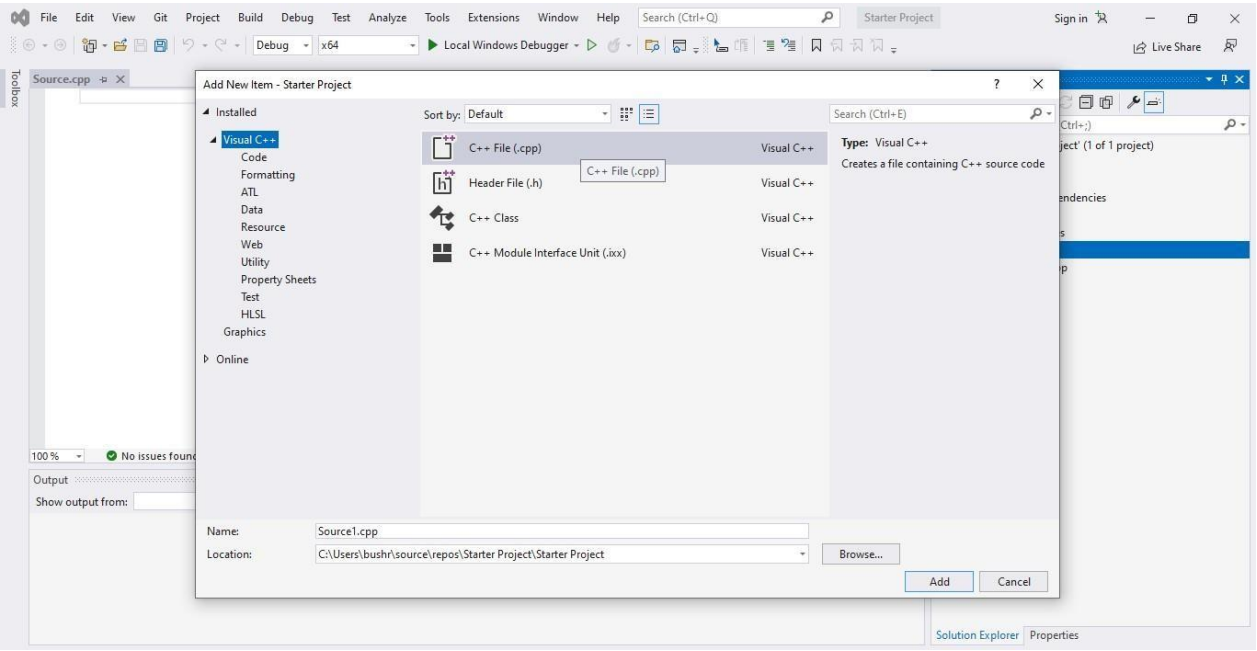
4. An empty C++ Windows console application gets created. Console applications use a Windows console window to display output and accept user input. In Visual Studio, an empty project will be created.



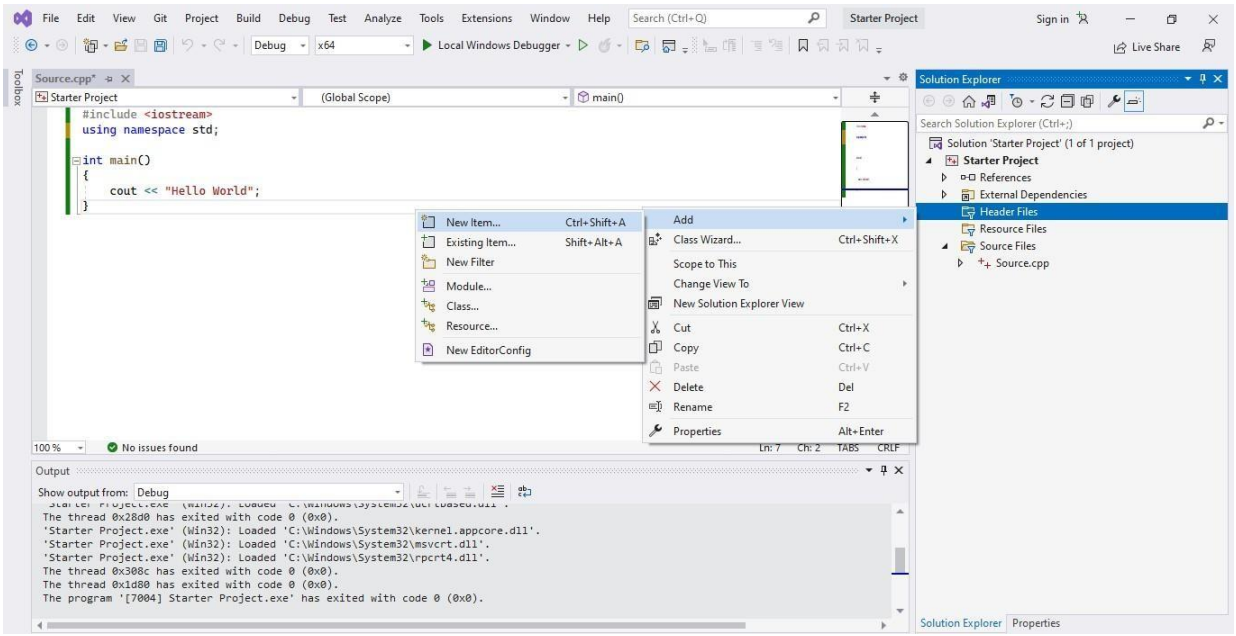
5. Right-click on Source Files and add a new item.



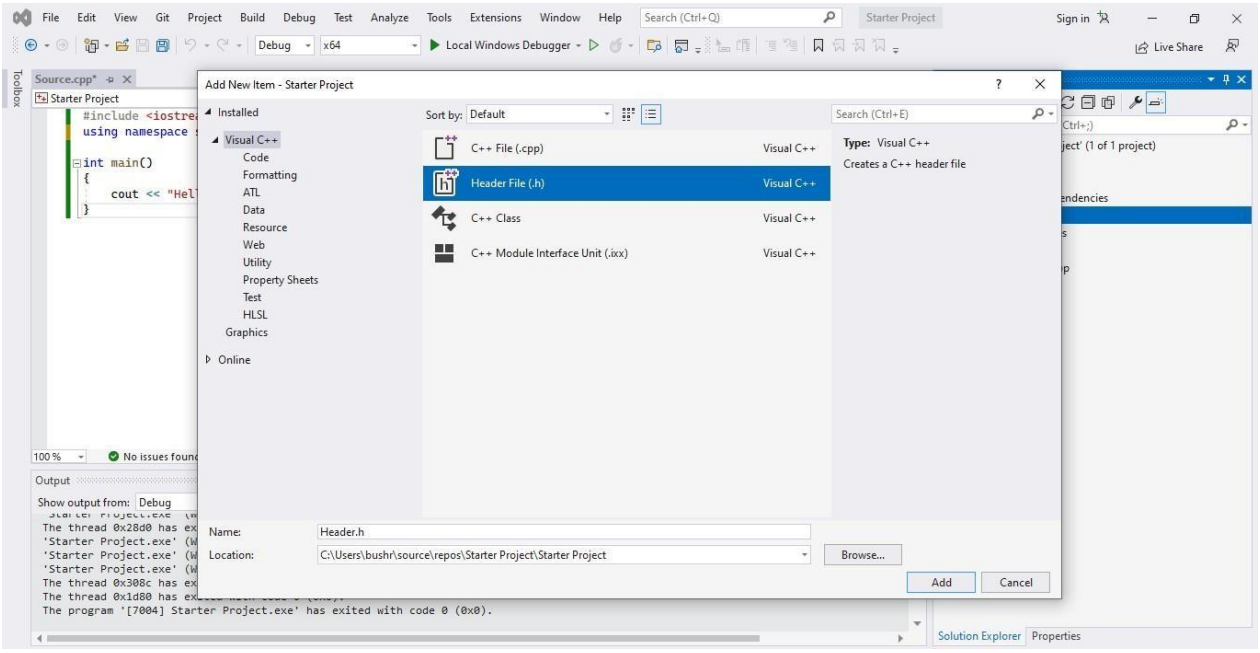
6. Create .cpp file and click Add



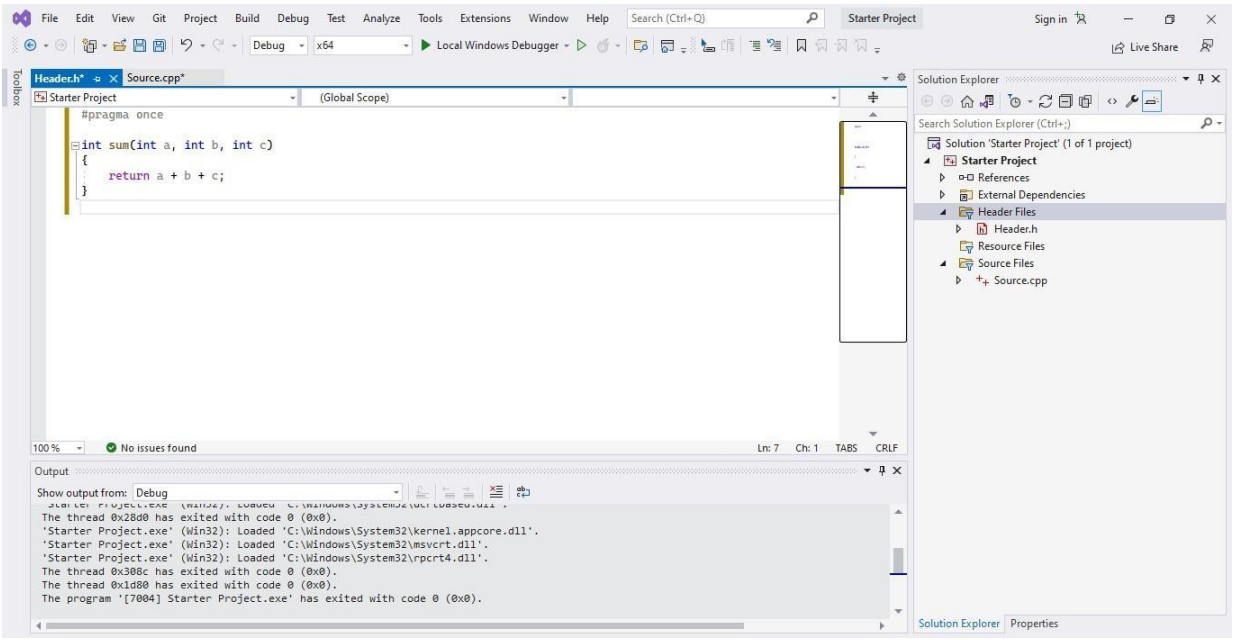
7. Right-click on Header Files and add a new item.



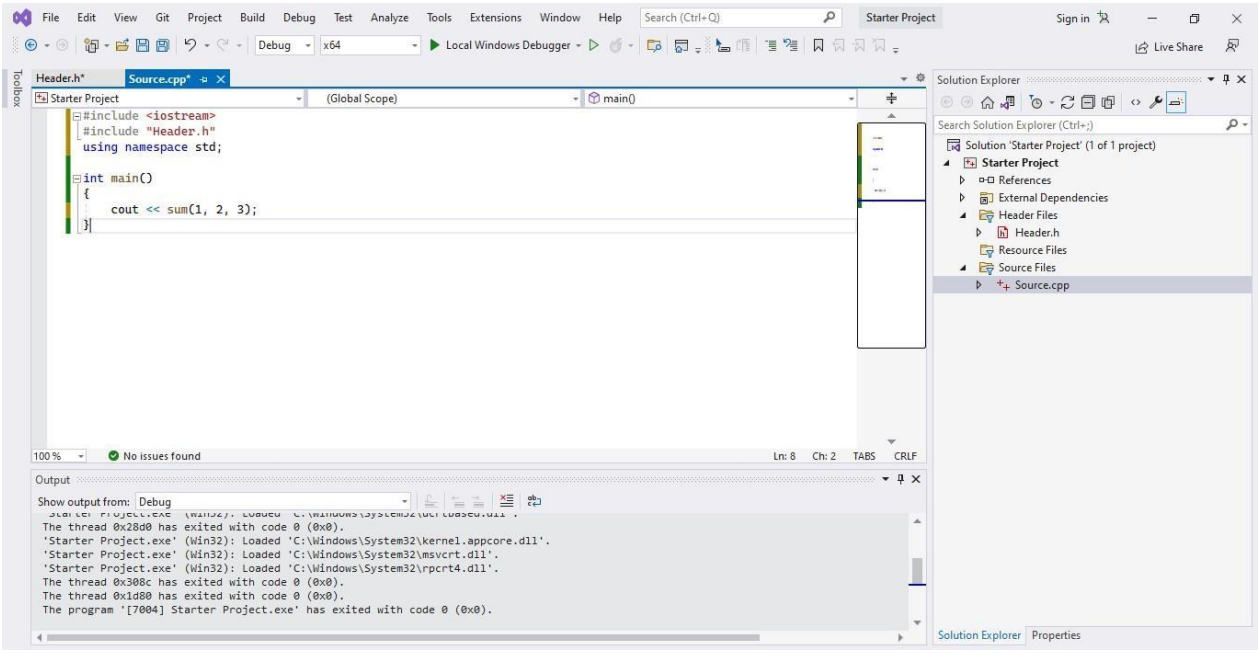
8. Select header file and click add



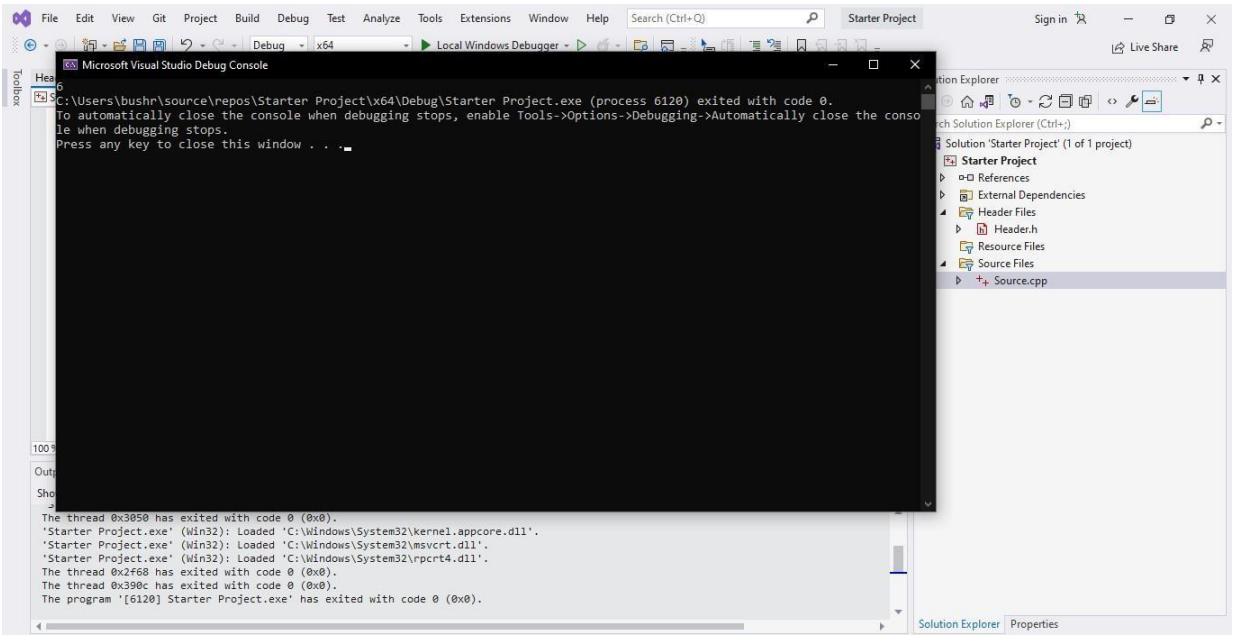
9. This is where you are going to write all your functions. Write sum() function in header file



10. You can use the functions written in the header file by including the file in Source.cpp.



11. Compile and run the code using the Local Windows Debugger option from the quick access toolbar followed by the green play button.



National University of computer and emerging sciences

Getting started with Gtest

Google test is a framework for writing C++ unit tests.

- When using googletest, you start by writing *assertions*, which are statements that check whether a condition is true.
- ***Tests*** use assertions to verify the tested code's behavior. If a test crashes or has a failed assertion, then it *fails*; otherwise it *succeeds*.
- A ***test suite*** contains one or many tests. You should group your tests into test suites that reflect the structure of the tested code.
- A ***test program*** can contain multiple test suites.

This is how a test suite looks like,

```
TEST_F(TestFixtureName, TestName) {
  ... test body ...
}
```

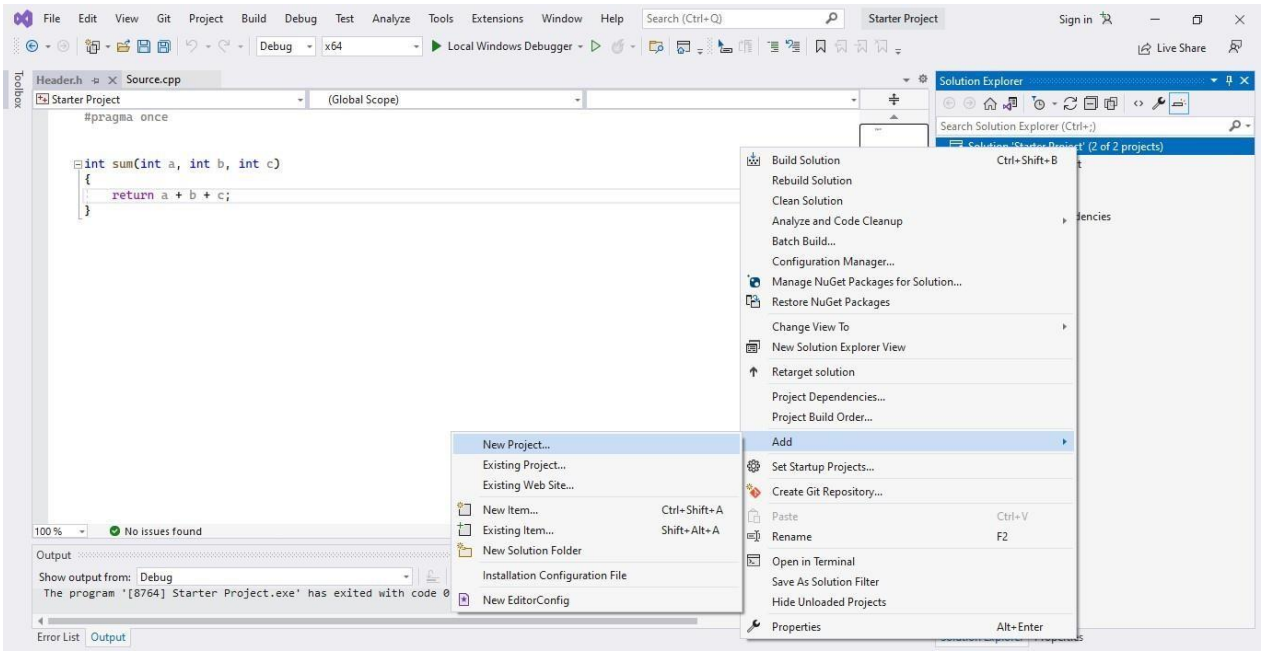
Example

```
// Tests factorial of positive numbers.
TEST(FactorialTest, HandlesPositiveInput) {
  EXPECT_EQ(Factorial(1), 1);
  EXPECT_EQ(Factorial(2), 2);
  EXPECT_EQ(Factorial(3), 6);
  EXPECT_EQ(Factorial(8), 40320);
}
```

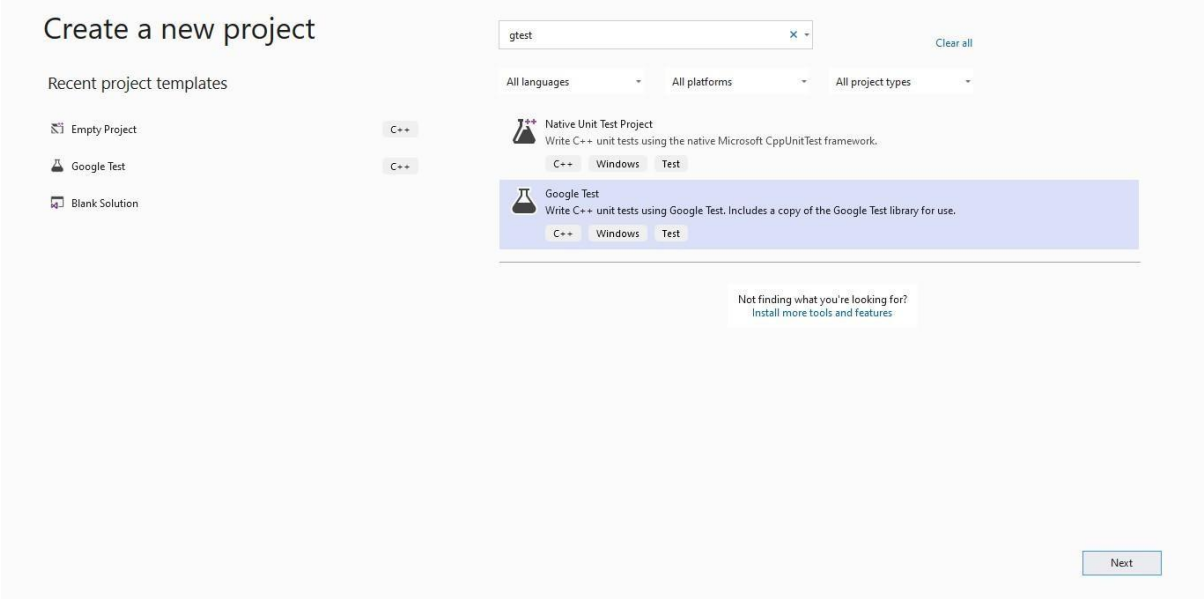
Visual Studio

In Visual Studio 2017 and later, Google Test is integrated into the Visual Studio IDE as a default component of the Desktop Development with C++ workload.

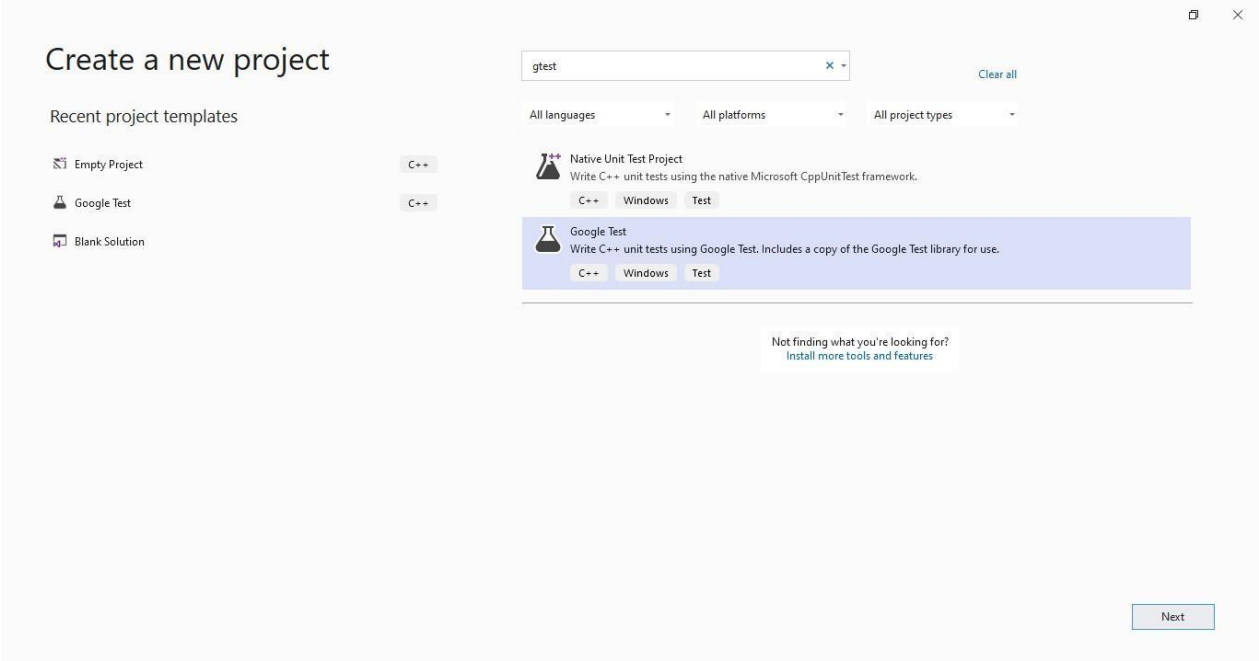
1. In Solution Explorer, right-click on the solution node and choose Add > New Project. Add a Google Test project in Visual Studio. Click on File and create a new project



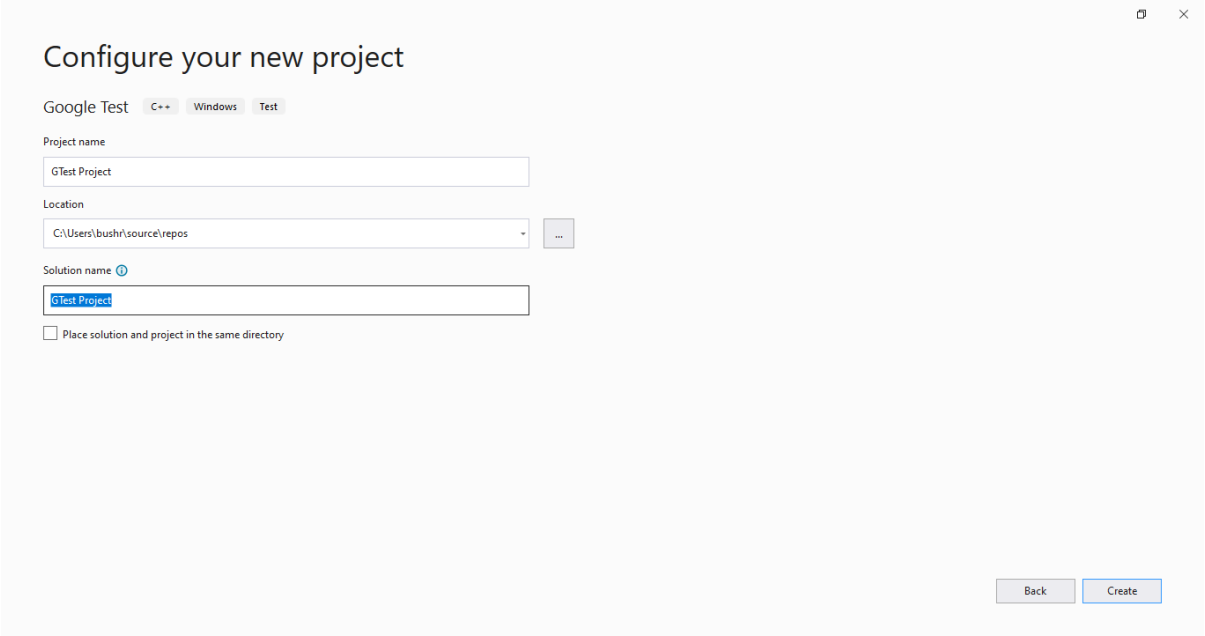
2. Search for google test and click next



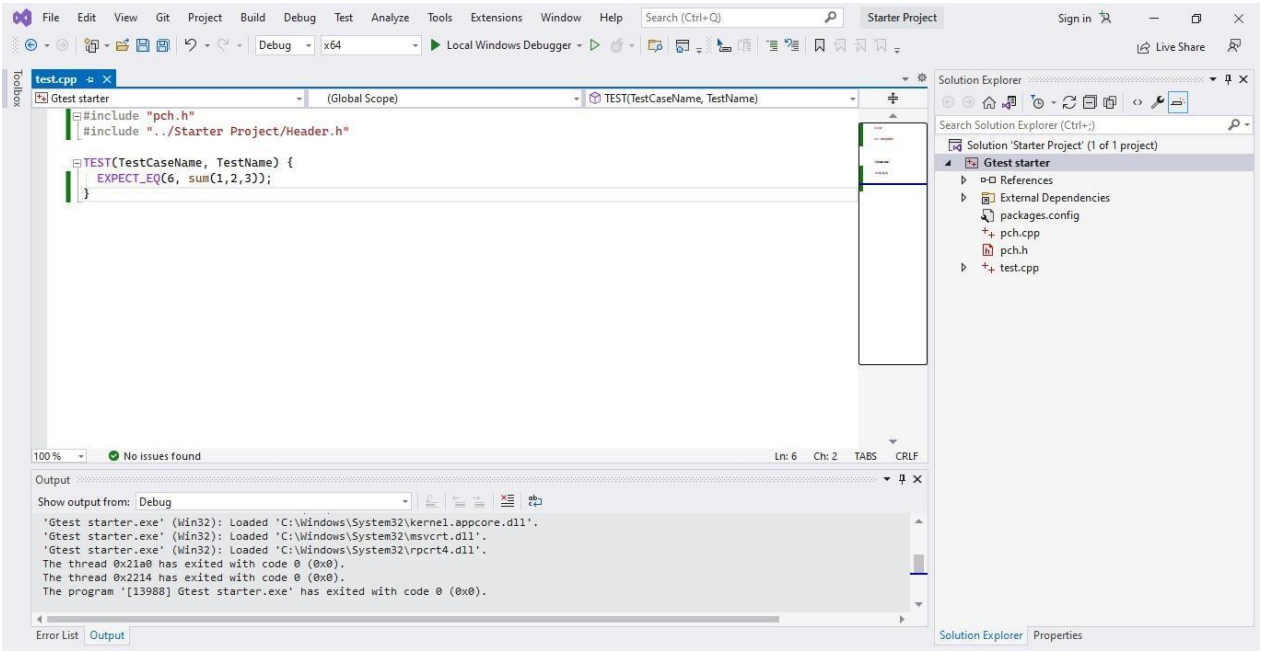
3. Rename the project and click create



4. Configure your project and click create

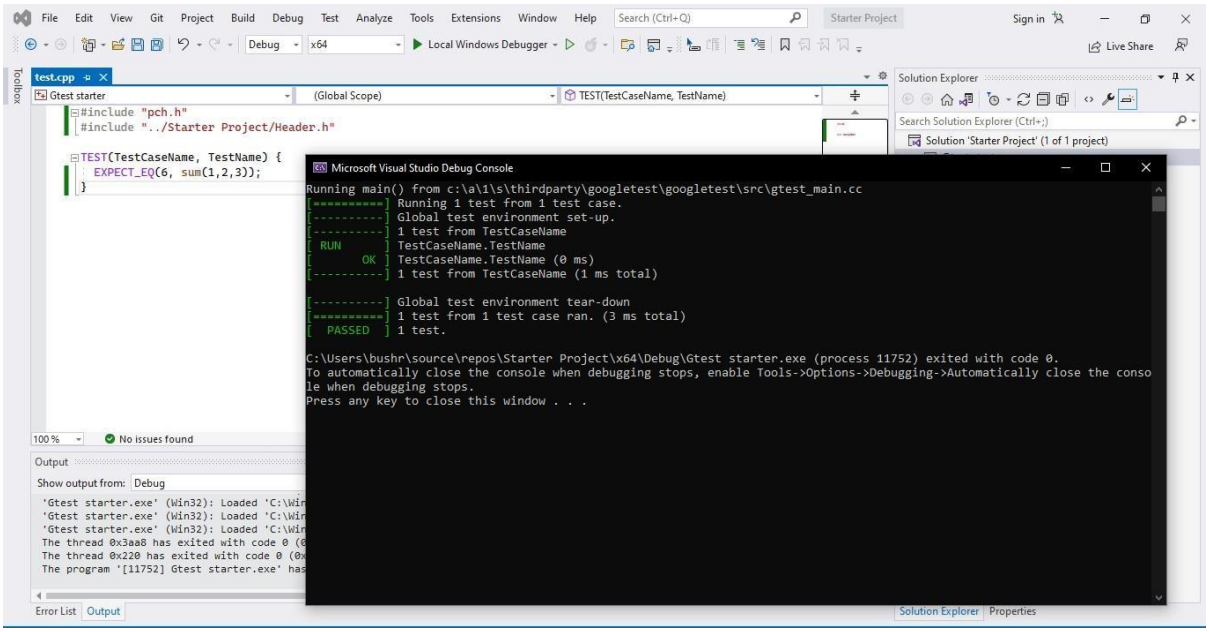


5. You are now ready to write and run Google Tests. Include the header file you want to test and write the test case

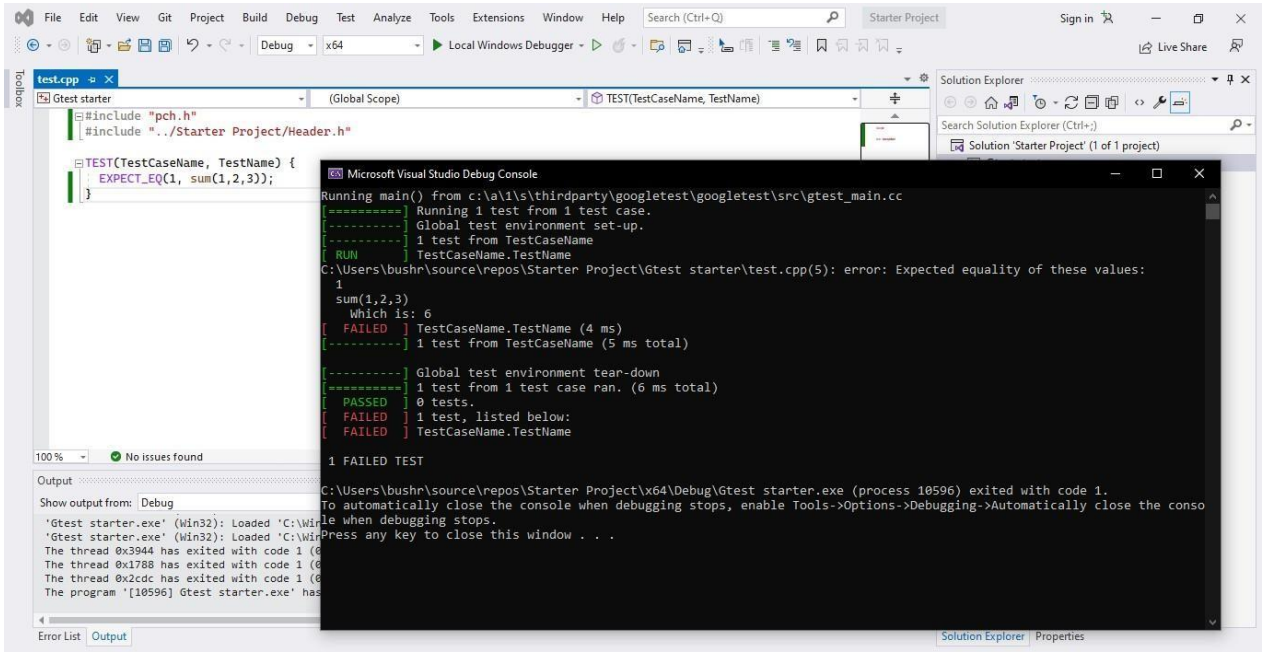


6. Compile and run the code using the Local Windows Debugger option from the quick access toolbar followed by the green play button

    a. Passed Test Case



b. Failed test case



National University
of computer and emerging sciences

Templates

Definition:

A **template in C++** is a feature that allows us to write **generic code** functions or classes that can work with any data type without rewriting the same logic for each type.

When a template is used, the compiler automatically generates a type specific version of the function or class for the data type provided.

In simple words:

Templates let us "write once, use for all types", while still keeping type safety and efficiency.

Why Templates?

Normally, when we write a function in C++, it only works for one specific data type. For example:

```
int addInt(int a, int b) { return a + b; }
double addDouble(double a, double b) { return a + b; }
```

Both functions do the same job, but we must repeat code just because the type is different. This quickly becomes boring, repetitive, and error-prone.

-) Templates solve this problem.

With templates, we can write a single generic function that works for all types:

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

Now, the same function works with int, double, char, even string:

```
cout << add(3, 5);           // int
cout << add(2.5, 4.7);       // double
cout << add('A', 'B');       // char
```

```
cout << add(string("Hi "), string("There"));  // string
```

Templates give us:

Reusability: one function for all types.

Type safety: compiler still checks types (if the data type are of

Efficiency: code is generated for each type at compile time.

Anatomy of a Function Template:

Consider this simple template:

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

Let's break it down word by word:

1. **Template**
   - This keyword tells the compiler: "I am defining a template (a blueprint), not a normal function."
2. **<typename T>**
   - Declares a template parameter named T.
   - T is a placeholder for a data type (like int, double, string).
   - Convention: use single capital letters (T, U, V), but you can choose any name.
3. **T add(...)**
   - The return type is also T.
   - Meaning: whatever type you pass in, that's the type you get back.
4. **(T a, T b)**
   - Function parameters are of type T.
   - When you call add(2, 3) , compiler substitutes T = int.
   - When you call add(2.5, 3.1) , compiler substitutes T = double.
5. **return a + b;**
   - The actual logic.
   - Works as long as the type T supports the + operator.

Rules of Type Deduction

- If both arguments are the **same type** , compiler deduces T automatically:

```
max(3, 7);        // T = int
max(2.5, 4.1);    // T = double
```

- If arguments are different but compatible (e.g., int and double), compiler cannot decide one T, error.

```
max(3, 4.5);   // ERROR: T = int? or T = double?
```

- Solutions:
  - Cast one argument, max<double>(3, 4.5)
  - Use multiple template parameters, template<typename T1, typename T2> auto max(T1 a, T2 b).

Two phase translation:

Templates are compiled in two passes. First, the compiler checks general syntax. Later, when you actually use the template with a type, it checks the type-specific parts. Errors can therefore appear later, only when you call the template with certain types.

Always put template function definitions in the header file (.h).

Template Argument Deduction:

- Compiler deduces T from the arguments.
- If both arguments are the same type , argument is deduced easily .
- If arguments are different types (int + double) → error, unless you:
  - cast one,
  - specify T explicitly, or
  - write a version with two template parameters.
- Some conversions (const, reference, array → pointer) happen automatically.
- Default template arguments are needed if no argument is passed at all.

Multiple Template Parameters:

So far, we used one template parameter:

```
template<typename T>
T max(T a, T b);   // both must be same type
```

But what if the two arguments are different types? E.g:

```
max(4, 7.2);   // int and double
```

A single T doesn't work here, so we allow two type parameters:

```
template<typename T1, typename T2>
T1 max(T1 a, T2 b) {
    return (b < a) ? a : b;
}
```

If you choose one argument type as the return type (like T1), the other argument gets converted sometimes in surprising ways.

Example:

```
max(66.66, 42);   // returns double 66.66
max(42, 66.66);   // returns int 66  Error
```

So the return type depends on argument order, which is dangerous.

Solutions in C++:

Explicit Return Type Parameter

You can add a third template parameter just for the return type:

```
template<typename T1, typename T2, typename RT>
RT max(T1 a, T2 b);
```

But since RT cannot be deduced from the call, you must specify it explicitly:

```
max<int,double,double>(4, 7.2);   // tedious
```

Example 1: Function templates for Addition

```cpp
#include <iostream>
using namespace std;

template <typename T>
T Add(T a, T b) {
    return a + b;
}

int main() {
    int sum_int = Add(5, 7);
    double sum_double = Add(3.14, 1.618);

    cout << "Sum (int): " << sum_int << endl;
    cout << "Sum (double): " << sum_double << endl;

    return 0;
}
```